

OmniAI

Complete Platform Guide

Every feature. Every API. Simply explained.

Milestones 1 through 20 -- Full Reference

Area	What you get
Document Ingestion	Upload PDFs, Word docs, HTML; automatic parsing and indexing
Smart Search	Vector + BM25 hybrid search, HyDE, reranking, streaming, tool calling
Chat & RAG	Multi-turn conversations with citations, forking, export
LLM Providers	Anthropic Claude, OpenAI GPT, Google Gemini, local Ollama
Agents	DAG-based AI workflows: fan-out, HITL, replay, templates, cost tracking
Code Sandbox	Run Python, JavaScript, Bash safely (subprocess / Docker / gVisor)
Deployments	Public chat pages and webhooks with rate limits and daily quotas
Connectors	Sync from Google Drive, SharePoint, Notion, Confluence, Slack, S3
Identity & Access	MFA (TOTP), OIDC (Google/GitHub/Microsoft), teams, RBAC, invitations
Observability	Cost dashboards, retrieval quality, audit logs, Prometheus, OpenTelemetry

Contents

1. Getting Started

- Installation & first run
- Default admin credentials
- Project layout

2. Core Concepts

- Tenants & multi-tenancy
- Collections, Documents, Chunks

3. Document Management

- Upload files
- Bulk operations
- Tags & reindex
- Knowledge graph

4. Search & Retrieval

- Basic semantic search
- BM25 keyword search
- HyDE expansion
- Streaming search
- Tool calling
- Multi-modal (images)
- Reranking
- Caching

5. Chat & Conversations

- Create & manage conversations
- Send messages with RAG
- Regenerate responses
- Fork conversations
- Export conversations

6. LLM Providers

- Anthropic Claude
- OpenAI GPT
- Google Gemini
- Ollama (local)

7. Agent Platform

- Create agents (DAG definition)
- Run agents
- Parallel fan-out / join
- Human-in-the-Loop (pause & resume)
- Time-travel replay
- Template marketplace

Multi-language code nodes
gVisor sandbox
Cost tracking & alerts
Export agent runs

8. Deployments

Public chat pages
Webhook deployments
Rate limits & quotas

9. Connectors

Local folder
Amazon S3
Google Drive
SharePoint
Notion
Confluence
Slack

10. Identity & Access

Registration & login
API keys
MFA (TOTP)
OIDC (Google / GitHub / Microsoft)
Teams & roles
Invitations
RBAC on collections

11. Observability

LLM cost dashboard
Retrieval quality metrics
Audit log
Prometheus metrics
OpenTelemetry tracing
Sentry

12. Security

Rate limiting
Account lockout
Encryption at rest
Environment security

13. Configuration Reference

All environment variables with defaults

14. API Quick Reference

All 80+ endpoints at a glance

1. Getting Started

What is OmniAI?

OmniAI is an **open-source, multi-tenant AI platform**. Think of it as a private ChatGPT that you run on your own computer or server -- one that can read *your* documents, remember conversations, and run automated AI workflows called **Agents**. Everything stays on your infrastructure. Nothing is sent to a third party unless you connect your own OpenAI or Anthropic key.

Key idea: You upload your documents (PDFs, Word files, web pages). OmniAI reads them, turns them into searchable 'chunks', and lets any AI model answer questions about them -- with references to the exact passages it used.

Installation (Developer Setup)

- **Requirements:** Python 3.11+, Node.js 18+, Git.
- **Clone the repo** and enter the backend folder.
- **Create a virtual environment** and install dependencies.
- **Copy** .env.example to .env (no changes needed for local dev).
- **Start the server** -- it auto-creates the database and default admin user on first run.

```
git clone https://github.com/your-org/omniai.git
cd omniai/backend
python -m venv .venv && .venv/Scripts/activate # Windows
pip install -e '.[dev]'
cp .env.example .env
python main.py # starts on http://localhost:9380
```

Docker Setup (Recommended for Teams)

```
cd omniai
docker compose up --build
# API at http://localhost:9380
# Frontend at http://localhost:5173
```

Default Admin Credentials

Setting	Default value
Email	admin@omniai.local
Password	Admin12345!
Tenant slug	local-dev
API base URL	http://localhost:9380
Docs (Swagger)	http://localhost:9380/docs

Security: Change these immediately before deploying to a server! Set `BOOTSTRAP_ADMIN_EMAIL`, `BOOTSTRAP_ADMIN_PASSWORD`, and `AUTH_SECRET` in your `.env` file.

Project Layout (Backend)

Folder	What lives here
omniai/application/	Business logic -- services that do the actual work
omniai/domain/	Data models -- the 'nouns' of the system (Document, Agent, etc.)
omniai/interfaces/	HTTP routes -- the URLs your browser or app calls
omniai/adapters/	Database, object storage, search engine connections
omniai/plugins/	Swappable components: parsers, LLMs, sandboxes, embeddings
omniai/config/	All settings -- read from environment variables
omniai/security/	Encryption, password hashing, RBAC permissions
omniai/observability/	Metrics, cost tracking, audit log, tracing
tests/	Automated tests (340+ passing)

2. Core Concepts

Tenants -- The Top-Level Container

Every piece of data in OmniAI belongs to a **Tenant**. A tenant is like a company account -- completely isolated from other tenants. When you run OmniAI for yourself, you have one tenant. If you host it for multiple teams, each team is a separate tenant.

Concept	Plain-English meaning
Tenant	Your organisation's private space -- nobody else can see your data
User	A person who logs in; belongs to one or more tenants
Role	OWNER / ADMIN / MEMBER -- controls what a user can do
Team	A sub-group of users within a tenant (for organising people)
API Key	A token (starts with omsk_) that lets code talk to OmniAI without logging in

Collections -- Folders for Your Documents

A **Collection** is a group of related documents -- like a folder. Each collection has its own AI embedding model (controls how documents are searched) and chunk template (controls how documents are split up).

Example: You might have a 'Legal Documents' collection and a 'Product Manuals' collection. They stay separate so searching legal questions only looks in legal docs.

Property	What it controls
name	Human-readable label
embedding_model	AI model that converts text to numbers (for search)
chunk_template	How documents are split: general / qa / sentence_window / small_to_big
system_prompt	Optional instructions given to the AI when chatting about this collection
top_k	How many chunks to retrieve per query (default: 5)
vector_weight	0.0 = pure keyword, 1.0 = pure vector, 0.5 = balanced (default)

Documents -- Your Files

A **Document** is any file you upload -- PDF, Word (.docx), HTML, plain text. After uploading, it goes through a pipeline:

1. **PENDING** -- Just uploaded, waiting in the queue.
2. **PARSING** -- Being read and converted to plain text.
3. **PARSED** -- Text extracted successfully.
4. **EMBEDDING** -- Text is being turned into search vectors.
5. **INDEXING** -- Vectors are being saved to the search engine.
6. **READY** -- Fully searchable! Queries will find it.
7. **FAILED** -- Something went wrong (check the error_message field).

Track progress: Call GET /v1/documents/{id}/status -- it returns a percentage (0-100%) and the current stage.

Chunks -- The Searchable Pieces

When a document is indexed, it is split into small **Chunks** -- usually a few hundred words each. The AI searches these chunks, not the whole document. This is what makes retrieval fast and precise.

Template	How it splits your document	Best for
general	Fixed 1024-character blocks with 100-char overlap	Most documents
qa	Q&A pairs (question / answer sections)	FAQ documents
sentence_window	Sentence-aware with surrounding context window	Precise retrieval
small_to_big	Small detail chunks + larger parent summaries	Long reports

3. Document Management

Uploading Documents

Upload a single file:

```
POST /v1/collections/{collection_id}/documents/upload
Content-Type: multipart/form-data
file=@your_document.pdf
```

Upload up to **20 files at once** (batch upload):

```
POST /v1/collections/{collection_id}/documents/bulk-upload
Content-Type: multipart/form-data
files[]=@doc1.pdf&files[]=@doc2.docx
```

Supported format	MIME type
PDF	application/pdf
Word document	application/vnd.openxmlformats-officedocument.wordprocessingml.document
HTML	text/html
Plain text	text/plain

Max file size: 100 MB per file (change with UPLOAD_MAX_BYTES env var). **Max documents per tenant:** 10,000 (change with TENANT_MAX_DOCUMENTS). **Max total storage:** 50 GB (change with TENANT_MAX_STORAGE_BYTES).

Checking Document Status

```
GET /v1/documents/{document_id}/status
```

```
// Response:
{ "status": "READY", "progress_pct": 100, "parser_name": "pdf", "page_count": 12 }
```

Bulk Operations -- Do Things to Many Documents at Once

Instead of deleting or reindexing documents one by one, use the bulk endpoint:

```
POST /v1/documents/bulk
{
  "document_ids": ["doc_abc", "doc_def", "doc_xyz"],
  "action": "delete" // options: delete | reindex | set_tags
  "tags": ["finance", "2025"] // only needed for set_tags
}
```

Action	What it does
delete	Permanently removes the documents AND their chunks from search
reindex	Re-reads and re-indexes the files (useful after changing embedding model)
set_tags	Attaches labels to all selected documents at once

Document Tags

Tags are labels you attach to documents to help organise and filter them.

```
PUT /v1/documents/{document_id}/tags
{ "tags": ["contract", "2025", "legal"] }
```

```
// Find all documents with a specific tag:
GET /v1/collections/{collection_id}/documents/by-tag/contract
```

Downloading & Reading Document Content

```
GET /v1/documents/{document_id}/raw // download original file bytes
GET /v1/documents/{document_id}/text // get the extracted plain text
GET /v1/documents/{document_id}/download-url // get a time-limited download link
```

Knowledge Graph

OmniAI automatically extracts **entities and relationships** from your documents and builds a knowledge graph. For example, from a legal document it might extract: *Acme Corp -- signed -- Contract 2025-001*.

```
GET /v1/documents/{document_id}/graph // triples in one document
GET /v1/collections/{collection_id}/graph // all triples in a collection
GET /v1/collections/{collection_id}/graph?entity=Acme Corp // filter by entity
```

Field	Meaning
subject	The main thing being talked about (e.g. 'Acme Corp')
predicate	The relationship (e.g. 'signed')
object	What it relates to (e.g. 'Contract 2025-001')
confidence	How sure the AI is (0.0 - 1.0)

4. Search & Retrieval

Retrieval is the heart of OmniAI -- it finds the most relevant chunks from your documents to answer a question. The system uses **hybrid search** by default: a combination of keyword search (BM25) and AI vector search.

Basic Search

```
POST /v1/retrieve
{
  "query": "What are the payment terms?",
  "collection_ids": ["col_abc", "col_def"], // search multiple collections
  "top_k": 8, // how many results to return
  "vector_weight": 0.7 // 0=keyword only, 1=vector only
}
```

Each result includes:

- **text** -- the actual passage found
- **score** -- relevance score (0 to 1, higher is better)
- **document_id / document_name** -- which document it came from
- **ordinal** -- position within the document

HyDE -- Smarter Search (Hypothetical Document Embeddings)

HyDE makes search much better. Instead of searching for your query directly, the AI first *imagines* a perfect answer, then searches for chunks that match that imagined answer. Particularly good for conceptual questions.

```
POST /v1/retrieve
{
  "query": "What causes hyperinflation?",
  "hyde": true,
  "hyde_model": "gpt-4o"    // which model writes the hypothetical answer
}
```

If HyDE fails (e.g. no LLM configured), the system silently falls back to normal search -- you never get an error, just standard results.

Streaming Search -- Results Appear Instantly

Instead of waiting for all results before showing anything, streaming returns each result the moment it is found -- one by one, in real time.

```
POST /v1/retrieve/stream
{ "query": "renewable energy trends", "top_k": 10 }

// Server-Sent Events stream:
data: { "text": "Solar capacity doubled in 2024...", "score": 0.95 }
data: { "text": "Wind installations rose by 30%...", "score": 0.91 }
data: [DONE]
```

Tool Calling -- AI Searches Automatically

Give the AI model a special 'tool' so it can search your documents *by itself* while answering. The AI decides when to search and what to search for.

```
POST /v1/retrieve/tool
{ "question": "Summarise our Q4 financials", "top_k": 8 }

// Response includes:
{ "answer": "...", "hits": [...], "tool_calls_made": 2 }
```

Multi-Modal Search (Text + Images)

Search using an image instead of text. Send a base64-encoded image and the system automatically uses the CLIP model to find visually or conceptually similar documents.

```
POST /v1/retrieve
{
  "query": "...", // base64 image
  "collection_ids": ["col_abc"]
}

// OmniAI auto-detects 'data:image/' prefix and routes to CLIP model
```

Install: pip install open-clip-torch torch Pillow to enable multi-modal embeddings. No config change needed -- the system detects image inputs automatically.

Reranking -- Better Result Ordering

After retrieval, a second AI pass **re-orders** results to put the most relevant ones first. Enable via the RERANKER_KIND environment variable:

RERANKER_KIND	How it works	Requirement
paired	Embeds (query, chunk) pairs and scores them	Nothing extra
sentence transformers	SentenceTransformers cross-encoder model	pip install sentence-transformers

Retrieval Caching

Identical queries return cached results instantly instead of re-running the full vector search. Set the TTL (time-to-live) in seconds:

```
RETRIEVAL_CACHE_TTL_SECONDS=300 # cache results for 5 minutes
# Set to 0 (default) to disable caching
```

Submit feedback to improve search quality over time:

```
POST /v1/observability/retrieval-feedback
{ "query_hash": "abc123", "chunk_id": "chunk_xyz", "rank": 1, "relevant": true }
```

5. Chat & Conversations

Conversations let you have a **multi-turn chat** with an AI that uses your documents as its knowledge base. Every AI answer includes **citations** -- references to the exact document passages it used.

Creating & Managing Conversations

```
POST /v1/conversations
{
  "title": "Q4 Financial Analysis",
  "collection_ids": ["col_legal", "col_finance"], // which docs to search
  "model_provider": "anthropic",
  "model_name": "claude-3-5-sonnet-20241022",
  "system_prompt": "You are a financial analyst. Be concise.",
  "top_k": 6,
  "temperature": 0.3
}
```

Operation	Endpoint
List all	GET /v1/conversations
Get one	GET /v1/conversations/{id}
Update	PATCH /v1/conversations/{id} (title, pinned, model, system_prompt)
Delete	DELETE /v1/conversations/{id}
List messages	GET /v1/conversations/{id}/messages

Sending a Message (with RAG)

When you send a message, OmniAI automatically:

1. Searches your linked collections for relevant chunks.
2. Passes those chunks as context to the AI model.
3. Streams the AI's response back to you in real time.
4. Records citations (which chunks were used and their scores).

```
POST /v1/chat
{
  "conversation_id": "conv_abc123",
  "content": "What were the main risks identified in the Q4 report?"
}

// Response is streamed (text/event-stream):
data: {"delta": "The main risks identified were..."}
data: {"delta": " supply chain disruptions and..."}
data: {"citations": [{"document_name": "Q4_Report.pdf", "text": "..."}]}
data: [DONE]
```

Regenerate a Response

Didn't like the answer? Ask the AI to try again, optionally with different settings:

```
POST /v1/chat/regenerate
{
  "conversation_id": "conv_abc123",
  "temperature": 0.8,
  "model_name": "gpt-4o" // try a different model
}
```

Forking -- Branch a Conversation

Create a copy of a conversation at any point and explore a different direction without losing your original chat. Like a 'Save Game' checkpoint.

```
POST /v1/conversations/{id}/fork
{
  "title": "Alternative analysis",
  "fork_at_message_id": "msg_abc123" // optional: fork from a specific message
}
```

Exporting Conversations

Download any conversation as a file you can read in any text editor or word processor:

```
GET /v1/conversations/{id}/export?format=markdown // nice formatted text
GET /v1/conversations/{id}/export?format=json // raw data for developers
```

6. LLM Providers

OmniAI supports multiple AI model providers. You register them once with your API key and then any conversation or agent can use them. All credentials are encrypted at rest.

Adding a Provider

```
POST /v1/providers
{
  "kind": "anthropic", // anthropic | openai | gemini | ollama
  "name": "My Claude",
  "credentials": { "api_key": "sk-ant-..." },
  "default_model": "claude-3-5-sonnet-20241022"
}
```

Provider	kind=	Popular models
Anthropic	anthropic	claude-3-5-sonnet-20241022, claude-3-opus-20240229
OpenAI	openai	gpt-4o, gpt-4o-mini, gpt-3.5-turbo
Google	gemini	gemini-1.5-pro, gemini-1.5-flash
Ollama	ollama	llama3.2, mistral, phi3 (runs on your machine)

Ollama -- Run AI Models Locally (Free)

Ollama lets you run AI models on your own computer for free -- no API key needed. Great for privacy-sensitive workflows.

```
# 1. Install Ollama from https://ollama.com
# 2. Pull a model:
ollama pull llama3.2

# 3. Register it in OmniAI:
POST /v1/providers
{ "kind": "ollama", "name": "Local LLaMA", "base_url": "http://localhost:11434" }
```

For embedding + OCR: Ollama also provides embedding models (nomic-embed-text) and vision models (llava) that OmniAI uses for indexing and OCR respectively.

Listing Available Models

```
GET /v1/providers/{provider_id}/models

// Returns list of model names the provider supports
```

Managing Providers

```
GET    /v1/providers           // list all registered providers
PATCH /v1/providers/{id}   // update credentials or default model
DELETE /v1/providers/{id}  // remove provider
```

7. Agent Platform

An **Agent** is an automated AI workflow. You design it as a graph (diagram) of steps -- called **nodes** -- connected by arrows. When you run the agent, it follows the graph, executing each node in order.

Think of it like a recipe: Node 1 = search documents. Node 2 = run a calculation. Node 3 = ask a human to approve. Node 4 = write the final answer. The agent follows every step automatically.

Creating an Agent

```
POST /v1/agents
{
  "name": "Contract Analyser",
  "description": "Searches legal docs and summarises key clauses",
  "definition": {
    "nodes": [
      { "id": "retrieve", "type": "retrieval",
        "collection_id": "col_legal", "top_k": 6 },
      { "id": "generate", "type": "llm",
        "prompt": "Summarise the key clauses: {{context}}" }
    ],
    "edges": [
      { "from": "retrieve", "to": "generate" }
    ]
  }
}
```

Node Types

Node type	What it does
retrieval	Searches a collection and adds results to context
llm	Calls an AI model with a prompt (uses {{context}} and {{user_input}})
code	Runs Python, JavaScript, or Bash code (requires sandbox enabled)
human_input	Pauses the run and waits for a human to approve or provide input
fan_out	Splits into multiple parallel branches (for doing things simultaneously)
join	Waits for all parallel branches to finish, then merges their results

Running an Agent

```
POST /v1/agents/{agent_id}/runs
{ "input": "Analyse the Acme Corp contract for payment risks" }

GET /v1/agents/{agent_id}/runs/{run_id}  // check status

// Status values: QUEUED -> RUNNING -> COMPLETED (or FAILED, PAUSED, CANCELLED)
```

Parallel Fan-Out / Join -- Do Multiple Things at Once

Add a **fan_out** node to split the workflow into parallel branches that run simultaneously, then a **join** node to merge all their results.

```
"nodes": [
  { "id": "fanout", "type": "fan_out" },
  { "id": "search_legal", "type": "retrieval", "collection_id": "col_legal" },
```

```

{ "id": "search_finance", "type": "retrieval", "collection_id": "col_finance" },
{ "id": "join", "type": "join" },
{ "id": "answer", "type": "llm", "prompt": "Summarise: {{context}}" }
],
"edges": [
{ "from": "fanout", "to": "search_legal" },
{ "from": "fanout", "to": "search_finance" },
{ "from": "search_legal", "to": "join" },
{ "from": "search_finance", "to": "join" },
{ "from": "join", "to": "answer" }
]

```

Result: Both searches run at the same time. When both finish, 'join' combines all found chunks and passes them to 'answer'. Twice as fast as running sequentially!

Human-in-the-Loop (HITL) -- Pause & Approve

Add a **human_input** node anywhere in your graph. When the agent reaches it, the run **pauses** and waits for a human to review and approve before continuing.

1. Agent reaches the **human_input** node.
2. Run status changes to **PAUSED**.
3. Your system polls the run status and shows it to a human reviewer.
4. Human calls the **resume** endpoint with their feedback.
5. Agent continues from the next node automatically.

```

POST /v1/agents/{agent_id}/runs/{run_id}/resume
{
  "human_input": "Approved. The payment terms look correct.",
  "approved": true
}

```

Time-Travel Replay -- Undo & Retry Any Step

Every agent run records a full **event log** -- every step is saved. You can 'rewind' to any past event and replay the run from that point, optionally with a different input. Creates a new run, preserving the original.

```

POST /v1/agents/{agent_id}/runs/{run_id}/replay
{
  "from_event": 3, // replay from event index 3
  "input_override": "Try a different approach" // optional new input
}

// Returns a new run_id. The new run fast-forwards through events 0-2,
// then continues LIVE from event 3.

```

Agent Template Marketplace

Ready-made agent blueprints you can import with one API call. No need to build from scratch -- pick a template and customise it.

```
GET /v1/marketplace/templates // browse available templates
GET /v1/marketplace/templates/{template_id} // see full definition

POST /v1/agents/import-template
{ "template_id": "parallel-research", "name": "My Research Agent" }
```

Template ID	What it does
basic-rag	Search a collection, then generate an answer. Simplest setup.
parallel-research	Search TWO collections simultaneously, then combine and answer.
human-review	Search, generate draft answer, pause for human approval, then send.
code-executor	Search context, then run a Python script with the results.
summarisation	Retrieve 10 passages from a document and write a full summary.

Import from your own URL (share templates with your team):

```
POST /v1/agents/import-template
{ "url": "https://your-server.com/my-template.json" }
```

Multi-Language Code Nodes

Code nodes can run **Python, JavaScript, or Bash**. The agent automatically injects the user's question and retrieved context as variables.

Language	Variables injected	Example use case
python	user_input, context_text	Data analysis, calculations, file processing
javascript	user_input, context_text	JSON manipulation, web logic
bash	\$USER_INPUT, \$CONTEXT_TEXT	Shell commands, file operations, system tasks

```
// JavaScript code node example:
const words = context_text.split(' ').length;
console.log(`Context has ${words} words.`);
```

Sandbox -- Safe Code Execution

Code nodes run inside a **sandbox** -- an isolated container that prevents malicious code from accessing your server, files, or secrets.

SANDBOX_KIND	Safety level	Requirement
none	Code nodes are disabled	Nothing -- safest option if you do not need code
subprocess	Runs in a separate Python process	Nothing extra
docker	Docker container isolation	Docker installed
gvisor	Intercepts all system calls (best)	gVisor runsc binary installed

```
SANDBOX_KIND=gvisor
GVISOR_RUNSC_BIN=/usr/local/bin/runsc
```

gVisor not installed? If the runsc binary is not found, OmniAI automatically falls back to subprocess mode with a log warning. Your agents still run -- just with less isolation.

Run Cost Tracking & Alerting

Every agent run shows how much it cost (estimated USD based on tokens used). Set a threshold -- if any run exceeds it, a warning appears in the logs.

```
AGENT_RUN_COST_ALERT_USD=0.10 # warn if a run costs more than $0.10

GET /v1/agents/{id}/runs/{run_id}
// Response includes: { "cost_usd": 0.0042, "status": "COMPLETED" }
```

Exporting Agent Runs

Download a full record of what the agent did -- every step, every result:

```
GET /v1/agents/{id}/runs/{run_id}/export?format=json // full event log
GET /v1/agents/{id}/runs/{run_id}/export?format=markdown // readable summary
```

8. Deployments -- Share AI with the World

A **Deployment** lets you publish a collection or agent as a public-facing chat page or webhook endpoint -- without requiring users to log in. Perfect for customer support bots, internal knowledge bases, or public Q&A; pages.

Creating a Deployment

```
POST /v1/deployments
{
  "name": "Customer Support Bot",
  "slug": "support", // your URL: /c/support
  "kind": "public_chat", // public_chat | webhook
  "target_kind": "collection", // collection | agent
  "target_id": "col_support_docs",
  "anonymous_allowed": true, // no login required
  "model_provider": "openai",
  "model_name": "gpt-4o-mini",
  "system_prompt": "You are a helpful customer support agent.",
  "rate_limit_per_minute": 20, // per visitor
  "daily_message_quota": 500 // max messages per day
}
```

Accessing the Public Chat Page

```
// Anyone (no login needed) can chat at:
GET /c/{slug} // loads the chat interface
POST /c/{slug}/chat // send a message
{ "content": "What is your return policy?" }
```


Rate Limits & Quotas

Setting	What it controls
rate_limit_per_minute	Max messages per visitor per minute (default: 20)
daily_message_quota	Max total messages across ALL visitors per day (default: 500)
anonymous_allowed	If false, visitors must log in before chatting

When a quota is exceeded: The API returns HTTP 429 (Too Many Requests) with a Retry-After header telling the client when to try again.

Managing Deployments

```
GET    /v1/deployments      // list all your deployments
GET    /v1/deployments/{id} // get status (message counts, today's usage)
PATCH /v1/deployments/{id} // update model, quota, system prompt, etc.
DELETE /v1/deployments/{id} // take it offline
```

9. Connectors -- Sync External Data Sources

Connectors automatically pull content from external platforms into your OmniAI collections. Once set up, they run on a schedule -- your knowledge base stays up to date without any manual uploads.

Available Connectors

kind	Source	What gets synced
local_folder	Local folder	All files in a directory on the server
s3	Amazon S3	Objects in an S3 bucket / prefix
gdrive	Google Drive	Docs, Sheets, Slides, PDFs in a folder
sharepoint	SharePoint Online	Documents in a SharePoint library
notion	Notion	Pages in a Notion workspace or database
confluence	Confluence	Pages in a Confluence space
slack	Slack	Messages and files from channels

Creating a Connector

```
POST /v1/connectors
{
  "name": "Legal Drive",
  "kind": "gdrive",
  "collection_id": "col_legal",
  "sync_interval_seconds": 3600, // sync every hour
  "config": {
    "folder_id": "1BxiMVs0XRA5nFMdKvBdBZjgmUUqptlbs740gVE2upms"
```

```
}  
}
```

Deduplication: Connectors track the SHA-256 hash of every file they sync. If a file has not changed since the last sync, it is skipped automatically -- no duplicate documents.

Connector Credentials (Environment Variables)

Connector	Required environment variable(s)
gdrive	GDRIVE_SERVICE_ACCOUNT_JSON (path to service account key file)
sharepoint	SHAREPOINT_TENANT_ID, SHAREPOINT_CLIENT_ID, SHAREPOINT_CLIENT_SECRET
notion	NOTION_API_KEY
confluence	CONFLUENCE_BASE_URL, CONFLUENCE_API_TOKEN
slack	SLACK_BOT_TOKEN
s3	OBJECT_STORE_ACCESS_KEY, OBJECT_STORE_SECRET_KEY, OBJECT_STORE_ENDPOINT

Triggering a Sync

```
POST /v1/connectors/{id}/sync // run sync immediately (don't wait for schedule)  
  
// Response:  
{ "discovered": 45, "ingested": 12, "skipped_duplicate": 33, "errors": 0 }
```

Preview Before Syncing

```
GET /v1/connectors/preview?connector_id={id}  
  
// Returns up to 20 items that WOULD be ingested -- without actually doing it  
// Great for verifying connector config before running for real
```

Managing Connectors

```
GET    /v1/connectors // list all connectors  
GET    /v1/connectors/{id} // get status (last_sync_at, last_synced_count, last_error)  
PATCH /v1/connectors/{id} // update config, sync_interval, or enabled flag  
DELETE /v1/connectors/{id} // remove connector (does NOT delete synced documents)
```

10. Identity & Access

Registration & Login

```
POST /v1/auth/register  
{ "email": "alice@example.com", "password": "MyStr0ng!Pass", "display_name": "Alice" }  
  
POST /v1/auth/login
```

```
{ "email": "alice@example.com", "password": "MyStr0ng!Pass" }

// Response: { 'token': 'eyJ...', 'mfa_required': false }
// Use: Authorization: Bearer eyJ... on all future requests
```

Security: Account logout: After 5 failed logins, the account is locked for 15 minutes. Configure with LOGIN_LOCKOUT_THRESHOLD and LOGIN_LOCKOUT_MINUTES.

Password Reset

```
POST /v1/auth/request-password-reset
{ "email": "alice@example.com" }

// In dev mode the response includes the token directly.
// In production: send the token to the user by email.

POST /v1/auth/reset-password
{ "token": "abc123xyz", "new_password": "NewStr0ng!Pass2" }
```

API Keys -- Code Access Without Passwords

For scripts and integrations, create an API key. It starts with omsk_ and is sent in the Authorization: Bearer omsk_... header.

```
POST /v1/api-keys
{ "name": "CI Pipeline", "scopes": ["read", "write"] }

// Response: { 'key': 'omsk_live_abc123...', 'id': 'key_xyz' }
// IMPORTANT: Copy the key now -- it won't be shown again!

POST /v1/api-keys/{id}/revoke // deactivate key
```

Multi-Factor Authentication (MFA / TOTP)

Enable TOTP-based two-factor authentication. Works with any authenticator app (Google Authenticator, Authy, 1Password, etc.).

1. Call POST /v1/auth/mfa/setup -- get a QR code URI.
2. Scan the QR code with your authenticator app.
3. Call POST /v1/auth/mfa/confirm with the 6-digit code to enable.
4. On future logins, the login response includes mfa_required: true.
5. Call POST /v1/auth/mfa/verify with the current 6-digit code to complete login.

```
POST /v1/auth/mfa/setup // { 'totp_uri': 'otpauth://...' }
POST /v1/auth/mfa/confirm { 'code': '123456' }
POST /v1/auth/mfa/verify { 'code': '654321' }
POST /v1/auth/mfa/disable { 'code': '123456' } // to turn off
```

OIDC -- Login with Google, GitHub, or Microsoft

Let users sign in with their existing Google, GitHub, or Microsoft account -- no password required.

Provider	env vars needed
Google	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
GitHub	GITHUB_CLIENT_ID, GITHUB_CLIENT_SECRET
Microsoft	MICROSOFT_CLIENT_ID, MICROSOFT_CLIENT_SECRET

```
GET /v1/auth/oidc/{provider}/login // redirect user here
GET /v1/auth/oidc/{provider}/callback // OAuth2 callback (set in provider dashboard)
```

Teams & Roles

```
POST /v1/teams { 'name': 'Legal Team', 'description': '...' }
GET /v1/teams // list teams

// Tenant roles: OWNER (full control) | ADMIN (manage users) | MEMBER (use features)
```

Inviting Users

Instead of sharing passwords, send email invitations. Each invitation expires after 72 hours (configurable).

```
POST /v1/auth/invite
{ "email": "bob@example.com", "role": "MEMBER" }

// Bob receives an email with a link. When he clicks it, his account is created
// and he is automatically added to your tenant.
```

Per-Collection Access Control (RBAC)

Restrict who can read or write specific collections -- even within the same tenant.

```
POST /v1/collections/{id}/members
{ "user_id": "user_bob", "role": "VIEWER" } // OWNER | EDITOR | VIEWER

GET /v1/collections/{id}/members // list who has access
```

11. Observability

LLM Cost Dashboard

Track exactly how much you are spending on AI calls -- broken down by model and time period.

```
GET /v1/observability/cost?days=30

// Response:
{ 'total_usd': 4.23, 'by_model': { 'gpt-4o': 3.10, 'claude-3-5-sonnet': 1.13 },
  'by_day': [ { 'date': '2025-04-01', 'usd': 0.42 }, ... ] }
```

Cost calculation: Prompt tokens: \$0.002 / 1K. Completion tokens: \$0.006 / 1K. Adjust with LLM_COST_PER_1K_PROMPT and LLM_COST_PER_1K_COMPLETION env vars to match your actual provider rates.

Retrieval Quality Metrics

```
GET /v1/observability/quality

// Response: { 'ndcg_at_10': 0.72, 'hit_rate': 0.88, 'feedback_count': 142 }

// Submit feedback (thumbs up/down) to improve the score:
POST /v1/observability/retrieval-feedback
{ "query_hash": "abc123", "chunk_id": "chunk_xyz", "rank": 1, "relevant": true }
```

Metric	What it means	Good value
NDCG@10	Normalised Discounted Cumulative Gain -- measures ranking quality	> 0.7
Hit rate	Fraction of queries where a relevant chunk appears in top-10	> 0.85

Audit Log

Every significant action (login, document delete, agent run, settings change) is recorded in an audit log. Only admins can view it.

```
GET /v1/admin/audit-events?limit=50&offset=0

// Each event: { 'action': 'document.delete', 'actor': 'alice@...',
//               'target_type': 'Document', 'target_id': 'doc_xyz',
//               'created_at': '2025-04-15T10:22:00Z' }
```

Prometheus Metrics

OmniAI exposes a `/v1/metrics` endpoint in Prometheus format. Plug it into Grafana for real-time dashboards.

```
GET /v1/metrics

# omniai_http_requests_total{method='POST',path='/v1/chat',status='200'} 1482
# omniai_http_request_duration_seconds_bucket{le='0.5'} 1305
# omniai_rate_limited_total{tenant='...'} 12
```

OpenTelemetry Distributed Tracing

Send traces to any OpenTelemetry-compatible backend (Jaeger, Honeycomb, Datadog, etc.):

```
OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4317
OTEL_SERVICE_NAME=omniai-api
```

Sentry Error Tracking

```
SENTRY_DSN=https://your-key@sentry.io/your-project
SENTRY_TRACES_SAMPLE_RATE=0.1 # capture 10% of traces
```

12. Security

Rate Limiting

Every API endpoint is protected by a token-bucket rate limiter. If a client sends too many requests, it gets an HTTP 429 response.

```
RATE_LIMIT_PER_MINUTE=120    # max requests per minute per key/IP (default: 120)

# Exempt paths: /v1/health, /v1/metrics (monitoring never gets locked out)
```

Account Lockout

```
LOGIN_LOCKOUT_THRESHOLD=5    # failed attempts before lockout (default: 5)
LOGIN_LOCKOUT_MINUTES=15     # how long to lock the account (default: 15)
```

Security HTTP Headers

All responses include hardened security headers automatically:

Header	What it does
X-Content-Type-Options	Prevents MIME-type sniffing attacks
X-Frame-Options: DENY	Blocks clickjacking (your page cannot be embedded in iframes)
Content-Security-Policy	Restricts which scripts and resources can run on the page
Referrer-Policy	Limits what URL info is sent to third parties
Strict-Transport-Security	Forces HTTPS in production (HSTS, 1-year max-age)

Encryption at Rest

All sensitive data (LLM provider API keys, connector credentials) is encrypted before being stored in the database, using AES-256 (Fernet symmetric encryption).

```
ENCRYPTION_KEY=your-32-byte-base64-key-here
# Generate a secure key: python -c 'from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())'
```

Environment Security Checklist

- Change AUTH_SECRET from the default (used for session tokens).
- Change ENCRYPTION_KEY from the default (used for stored credentials).
- Change admin email and password from defaults.
- Set REGISTRATION_OPEN=false in production if you do not want public sign-ups.
- Use HTTPS (TLS) in front of OmniAI in production.
- Never commit your .env file to version control.

13. Configuration Reference

All settings are read from environment variables (or a .env file in the backend directory). Values shown are the defaults.

Application

Variable	Default	Notes
APP_NAME	Omni-AI	Name shown in UI and docs
APP_ENV	development	Set 'production' to enable HSTS etc.
HTTP_PORT	9380	Port the API server listens on
API_CORS_ORIGINS	http://localhost:5173	Comma-separated allowed origins
REGISTRATION_OPEN	true	false = no public sign-up

Database & Storage

Variable	Default	Notes
DB_URL	sqlite:///./omniai-dev.db	PostgreSQL in prod
AUTO_CREATE_SCHEMA	true	Run migrations on start
OBJECT_STORE_KIND	local	local s3
OBJECT_STORE_LOCAL_DIR	./omniai-storage	Where files are saved
OBJECT_STORE_BUCKET	omniai	S3 bucket name
UPLOAD_MAX_BYTES	104857600	100 MB per file
TENANT_MAX_DOCUMENTS	10000	Per-tenant doc limit
TENANT_MAX_STORAGE_BYTES	53687091200	50 GB per tenant

Search & Retrieval

Variable	Default	Notes
SEARCH_KIND	memory	memory pgvector pinecone weaviate
PGVECTOR_TABLE	omniai_vectors	PostgreSQL table name
PINECONE_API_KEY	(none)	Pinecone cloud key
PINECONE_ENVIRONMENT	us-east-1-aws	Pinecone region
WEAVIATE_URL	http://localhost:8080	Weaviate server URL
WEAVIATE_API_KEY	(none)	Weaviate auth (optional)
RETRIEVAL_CACHE_TTL_SECONDS	0	0 = disabled
RERANKER_KIND	(none)	paired sentencetransformers

LLM & OCR

Variable	Default	Notes
----------	---------	-------

OLLAMA_BASE_URL	http://localhost:11434	Ollama server address
OCR_KIND	none	none tesseract ollama_vision
OCR_MIN_CHARS_PER_PAGE	50	Pages with fewer chars get OCR'd
OCR_IMAGE_DPI	200	DPI for image extraction
OLLAMA_VISION_MODEL	llava	Model used for OCR

Authentication & Sessions

Variable	Default	Notes
AUTH_SECRET	change-me-...	MUST change in production
ENCRYPTION_KEY	dev-only-...	MUST change in production
SESSION_TTL_MINUTES	480	8-hour sessions
SESSION_COOKIE_NAME	omniai_session	Cookie name for browser sessions
LOGIN_LOCKOUT_THRESHOLD	5	Failed logins before lockout
LOGIN_LOCKOUT_MINUTES	15	Lockout duration
INVITATION_EXPIRY_HOURS	72	Invitation link lifetime

Agent Platform

Variable	Default	Notes
SANDBOX_KIND	none	none subprocess docker gvisor
SANDBOX_DEFAULT_TIMEOUT	30.0	Seconds before code is killed
GVISOR_RUNSC_BIN	runsc	Path to gVisor binary
AGENT_RUN_COST_ALERT_USD	0.0	0 = disabled
AGENT_MARKETPLACE_URL	https://marketplace.omniai.dev	Custom template registry

Observability

Variable	Default	Notes
OTEL_EXPORTER_OTLP_ENDPOINT	(none)	OpenTelemetry collector URL
OTEL_SERVICE_NAME	omniai-api	Service name in traces
SENTRY_DSN	(none)	Sentry project DSN
SENTRY_TRACES_SAMPLE_RATE	0.1	Fraction of traces sent
LLM_COST_PER_1K_PROMPT	0.002	USD per 1K prompt tokens
LLM_COST_PER_1K_COMPLETION	0.006	USD per 1K completion tokens

14. API Quick Reference

All endpoints require Authorization: Bearer {token} unless marked public. Base URL: <http://localhost:9380>

Auth & Users

Endpoint	Description
POST /v1/auth/register	Register new user
POST /v1/auth/login	Login (returns token)
POST /v1/auth/logout	Revoke session
GET /v1/auth/me	Get current user profile
POST /v1/auth/request-password-reset	Start password reset
POST /v1/auth/reset-password	Complete password reset
POST /v1/auth/mfa/setup	Generate TOTP QR code
POST /v1/auth/mfa/confirm	Enable MFA
POST /v1/auth/mfa/verify	Verify TOTP on login
POST /v1/auth/mfa/disable	Disable MFA
GET /v1/auth/oidc/{provider}/login	Start OIDC login
GET /v1/auth/oidc/{provider}/callback	OIDC callback
POST /v1/api-keys	Create API key
GET /v1/api-keys	List API keys
POST /v1/api-keys/{id}/revoke	Revoke API key

Collections & Documents

Endpoint	Description
GET /v1/collections	List collections
POST /v1/collections	Create collection
GET /v1/collections/{id}	Get collection
PATCH /v1/collections/{id}	Update collection
DELETE /v1/collections/{id}	Delete collection
GET /v1/collections/{id}/graph	Knowledge graph triples
GET /v1/collections/{id}/members	List access members
POST /v1/collections/{id}/members	Add member (RBAC)
GET /v1/collections/{id}/documents	List documents
POST /v1/collections/{id}/documents/upload	Upload single file
POST /v1/collections/{id}/documents/bulk-upload	Batch upload (max 20)
POST /v1/documents/bulk	Bulk delete / reindex / tag

GET	/v1/documents/{id}	Get document
DELETE	/v1/documents/{id}	Delete document
GET	/v1/documents/{id}/status	Indexing progress
GET	/v1/documents/{id}/text	Extracted text
GET	/v1/documents/{id}/download-url	Pre-signed download URL
GET	/v1/documents/{id}/graph	Document knowledge graph
PUT	/v1/documents/{id}/tags	Set document tags
POST	/v1/documents/{id}/reindex	Re-index document

Retrieval & Chat

Endpoint	Description
POST /v1/retrieve	Hybrid search (vector + BM25 + HyDE)
POST /v1/retrieve/stream	Streaming search (SSE)
POST /v1/retrieve/tool	AI-driven tool-call retrieval
GET /v1/chunks/{id}	Get single chunk
GET /v1/documents/{id}/chunks	List document chunks
GET /v1/conversations	List conversations
POST /v1/conversations	Create conversation
GET /v1/conversations/{id}	Get conversation
PATCH /v1/conversations/{id}	Update conversation
DELETE /v1/conversations/{id}	Delete conversation
GET /v1/conversations/{id}/messages	List messages
POST /v1/chat	Send message (streaming)
POST /v1/chat/regenerate	Regenerate last response
GET /v1/conversations/{id}/export	Export as JSON or Markdown
POST /v1/conversations/{id}/fork	Fork conversation

Agents & Marketplace

Endpoint	Description
GET /v1/agents	List agents
POST /v1/agents	Create agent
GET /v1/agents/{id}	Get agent
PATCH /v1/agents/{id}	Update agent
DELETE /v1/agents/{id}	Delete agent
POST /v1/agents/{id}/runs	Start run

GET	/v1/agents/{id}/runs	List runs
GET	/v1/agents/{id}/runs/{rid}	Get run status / output
POST	/v1/agents/{id}/runs/{rid}/resume	Resume PAUSED run (HITL)
POST	/v1/agents/{id}/runs/{rid}/replay	Replay travel replay
DELETE	/v1/agents/{id}/runs/{rid}	Cancel run
GET	/v1/agents/{id}/runs/{rid}/export	Export run log
GET	/v1/marketplace/templates	Browse templates
GET	/v1/marketplace/templates/{id}	Get template definition
POST	/v1/agents/import-template	Import from marketplace or URL

Providers, Connectors, Deployments

Endpoint	Description
GET /v1/providers	List providers
POST /v1/providers	Add provider (Anthropic/OpenAI/etc.)
PATCH /v1/providers/{id}	Update credentials or model
DELETE /v1/providers/{id}	Remove provider
GET /v1/providers/{id}/models	List available models
GET /v1/connectors	List connectors
POST /v1/connectors	Create connector
PATCH /v1/connectors/{id}	Update connector
DELETE /v1/connectors/{id}	Delete connector
POST /v1/connectors/{id}/sync	Trigger immediate sync
GET /v1/connectors/preview	Preview sync (dry run)
GET /v1/deployments	List deployments
POST /v1/deployments	Create public chat / webhook
PATCH /v1/deployments/{id}	Update deployment
DELETE /v1/deployments/{id}	Delete deployment
GET /c/{slug}	Public chat page (no auth)
POST /c/{slug}/chat	Send message to public deployment

Teams, Admin & Observability

Endpoint	Description
GET /v1/teams	List teams
POST /v1/teams	Create team
GET /v1/tenants/current	Current tenant info

GET /v1/admin/users	List all users (admin only)
GET /v1/admin/audit-events	Audit log (admin only)
GET /v1/observability/cost	LLM cost dashboard
GET /v1/observability/quality	Retrieval quality (NDCG, hit-rate)
POST /v1/observability/retrieval-feedback	Submit search relevance feedback
POST /v1/sandbox/run	Run code directly (dev/testing)
GET /v1/health	Health check (no auth needed)
GET /v1/metrics	Prometheus metrics (no auth needed)